

程序运行效率与时空复杂度

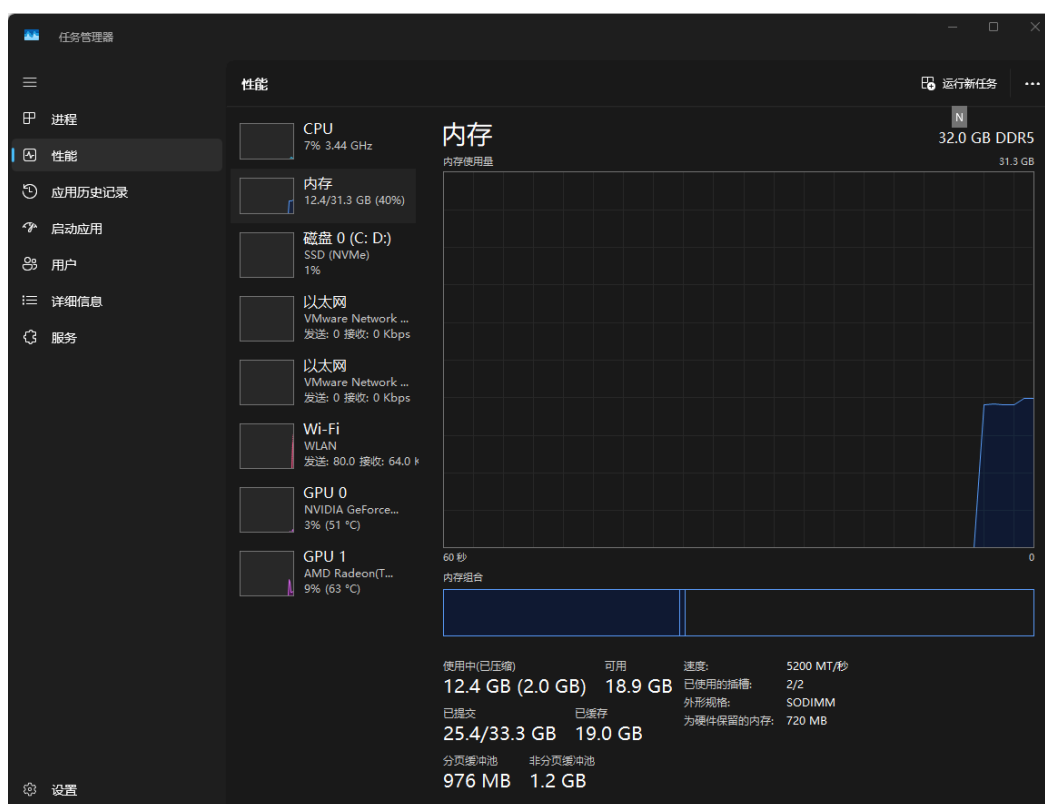
ExplodingKonjac

前言

计算机的运行速度很快，每秒能够进行大量的运算。但是计算机的速度也并非无限的。我们的 CPU 型号一般都会显示一个类似 3.5GHz 的参数，这个参数指的是 CPU 的时钟频率，比如这里就是说 CPU 一秒可以执行 3.5×10^9 个时钟周期。对于时钟周期是什么你不需要深入了解，你需要知道，CPU 执行指令、进行运算消耗的时间都以时钟周期为单位，所以时钟频率越高，CPU 的速度越快。

计算机同样还有内存（运行内存）的概念。这里纠正一个常见的误区，很多人经常说的“内存空间”，比如手机的“512GB 内存”，实际上说的是“硬盘空间”，也就是大家的电脑的 C 盘 D 盘下面显示的那个“剩余空间大小”的“空间”。而“内存”实际上的概念是“运行内存”，也就是计算机运行的时候，能留出多大的空间来给软件实时地使用。

如果你使用的是 Windows 系统，打开“任务管理器”，调到“性能”标签页，就可以看到电脑的 CPU 和内存信息：



那么，我们要衡量一个程序的性能，就有两方面：这个程序运行需要多少时间？这个程序运行需要消耗多少内存？这就是时间复杂度和空间复杂度在刻画的问题。

从『量级』谈起

对于两个函数 $f(x)$ 和 $g(x)$ ，我们希望比较它们在 x 足够大时大小。但是很多时候我们并不需要严格地进行比较。如果 $f(x) = 2^x$ ， $g(x) = x^2$ ，那么显然， $f(x)$ 的“量级”是远远大于 $g(x)$ 的“量级”的。

为了表示量级，我们引入大 O 表示法。对于一个函数 $f(x) : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ ，若在 x 充分大的时候， $f(x)$ 的上界的量级为 $g(x)$ ，那么记做 $f(x) = O(g(x))$ 。形式化一点来说，若

$$\exists C > 0 : \exists X > 0 : \forall x > X : f(x) \leq C \cdot g(x)$$

那么就可以记做 $f(x) = O(g(x))$ 。

举一个简单的例子，设 $f(x) = \frac{1}{2}x^2$, $g(x) = x \log_5 x$ ，那么我们可以写出：

$$f(x) = O(x^2), \quad g(x) = O(x \log x)$$

注意这里 $\log$ 没有底数。这里包含两个原因：

1. 计算机中， $\log$ 通常以 2 为底数，我们写的 $\log$ 默认表示 $\log_2$ 。
2. 根据换底公式， $\log_a x = \frac{\log_2 x}{\log_2 a}$ ，其中 $\log_2 a$ 是一个常系数，所以 $\log_a x = O(\log_2 x)$ ，因此底数在量级判断时是不重要的。

很明显在量级上， x^2 比 $x \log x$ 大，那么我们就可以立刻判断出来，在 x 足够大的时候，是有 $f(x)$ 远大于 $g(x)$ 的。

时间复杂度

引入

对于一个程序，定义其问题规模参数为 n （比如，求 n 个整数的和，算 $n \times n$ 的矩阵的乘法，等等），记程序的运行时间为函数 $T(n)$ 。

那么 $T(n)$ 似乎可以表示程序的运行时间了？但是这存在很多问题：

- $T(n)$ 本身没有一个解析的数学表达式，难以组合运算；
- $T(n)$ 和机器性能是相关的，在你的电脑上可能有 $T(100) = 0.1s$ ，但是在 10 年前的电脑上也许 $T(100) = 10s$ ；
- $T(n)$ 不够“纯粹”，不像是在描述一个“算法”的性质。

所以我们试图取消掉不同运行环境造成的限制。很好理解，虽然不同机器上程序的运行时间不一样，但是运行时间随着问题规模 n 增长的速率应该是差不多的。比如求 n 个数的和，在任何机器上，求 1000 个数的和所需要的时间应该都是求 100 个数的和所需要的时间的约 10 倍。也就是说，在“量级”的意义上，不同机器上的 T 应该是相等的，只有某个常数系数的差别：

$$T_1(n) \sim T_2(n) \sim T_3(n) \sim \dots$$

现在我们可以用上之前的大 O 表示法了。我们取一个函数 $f(n)$ 使得：

$$T_i(n) = O(f(n)) \quad i = 1, 2, \dots$$

那么我们就称 $O(f(n))$ 为该程序的**渐进时间复杂度**，简称**时间复杂度**。这代表着，程序的运行时间的量级，以及随着 n 扩大程序运行时间增长的速率，是符合 $f(n)$ 的特性的。

时间复杂度的计算

讲了很多理论上的东西，是时候进入实践了。计算时间复杂度，其实就是在估计程序执行的指令数量的量级，具体的表现就是程序进行了多少次循环、递归迭代了多少次等。让我们考察一个最简单的问题，计算 n 个数的和：

```

1  int sum = 0;
2  for (int i = 1; i <= n; i++) {
3      sum += a[i];
4  }

```

这里有一个循环，循环体会执行 n 次。循环体内的语句 `sum += a[i]` 具体会被拆成多少个 CPU 指令我们不知道，但是我们不关心，因为指令的数量（当然）是一个确定的、有界的常数（或者说，`sum += a[i]` 这个语句的时间复杂度是 $O(1)$ ）。假设循环体要执行 k 个指令，那么总指令数就是 kn 。而 k 是常数，它的量级只由 n 决定，所以我们可以得到这个程序的时间复杂度是 $O(n)$ 。

当然，当你理解并熟练了之后，你就不需要每次都进行这么多分析了，看到这样一个循环，你就能轻松反应过来时间复杂度是 $O(n)$ 。

接下来是一个更复杂一点的问题，下面是一个选择排序的算法：

```

1  for (int i = 1; i <= n; i++) {
2      for (int j = i + 1; j <= n; j++) {
3          if (a[i] < a[j]) {
4              int t = a[i];
5              a[i] = a[j];
6              a[j] = t;
7          }
8      }
9  }

```

先看最内层的 `if` 块，里面没有任何循环了，所以指令数量肯定是有确定的常数上界的，复杂度是 $O(1)$ 。而这个 `if` 块被包在了两层循环之内。我们可以简单计算一下这个双层循环会被执行多少次：

$$\sum_{i=1}^n \sum_{j=i+1}^n 1 = \sum_{i=1}^n (n-i) = \frac{n(n-1)}{2}$$

那么我们可以说，该程序的时间复杂度是 $O(\frac{n(n-1)}{2})$吗？

我们把这个式子拆开： $\frac{n(n-1)}{2} = \frac{1}{2}(n^2 - n)$ 。这个式子外面有一个 $\frac{1}{2}$ 的常数，然而我们在乎的是量级，常数系数是不重要的，因此可以去除。后面还有一个 $-n$ 项。显然在 n 很大的时候 n 远远小于 n^2 ，所以这里的 n 也不会影响量级，应该去除。最后化简得到这个程序的时间复杂度为 $O(n^2)$ 。

根据上面的讲解，大家应该对程序时间复杂度的计算有一定概念了，可以自己计算一下自己写过的代码的时间复杂度。

时间复杂度的应用

时间复杂度刻画的是程序运行的时间量级，因此当然可以用时间复杂度来估计程序运行的时间。

对于一个时间复杂度为 $O(f(n))$ 的程序，我们将 n 的最大可能数值 N 代入求得 $f(N)$ 。那么一个经验公式就是，程序运行时间所需要的秒数近似为 $f(N)/10^8$ 。

假设现在有一个时间限制为 1s 的题， $n \leq 10^5$ 。小 A 写了下面的代码：

```

1  int ans = 0;
2  for (int i = 1; i <= n; i++) {
3      int sum = 0;
4      for (int j = i; j <= n; j++) {
5          sum += a[j];
6          if (sum > ans) ans = sum;
7      }
8  }

```

而小 B 写了下面的代码：

```

1  int ans = 0, mn = 0, sum = 0;
2  for (int i = 1; i <= n; i++) {
3      sum += a[i];
4      if (sum - mn > ans) ans = sum - mn;
5      if (sum < mn) mn = sum;
6  }

```

这两个代码事实上都是等价的，但是运行时间显然不一样。根据之前讲解的知识，小 A 的代码时间复杂度为 $O(n^2)$ ，而小 B 的代码时间复杂度为 $O(n)$ 。代入 n 的最大值 10^5 ，得到 $(10^5)^2 = 10^{10}$ ，远远超出了 10^8 ，这意味着小 A 的代码无法通过本题，会获得 TLE 的结果。而小 B 的代码代入复杂度计算得到 $10^5 < 10^8$ ，可以轻松通过。

当然，这种运算仅仅是估测。即使对于时间复杂度相同的算法，其实际运行时间也有可能差异很大。主要有下面的原因：

1. 不同的运算操作消耗的时间不同。比如 CPU 执行乘法操作是比执行加法操作慢很多的，而执行内存读写则比运算都要慢。在这种情况下，即使程序运行时间显示出相同的量级，其在给定数据规模下表现也很不一样。我们称这个影响程序运行时间的常系数为算法复杂度的**常数**（简称算法的常数）。大家经常说的“常数大”“常数小”“卡常”就是这个含义。
2. 复杂度的上界难以取到。大 O 表示法给出的时间复杂度是程序实际运行时间的上界。但是现实问题中，有的算法实际运行性能常常难以达到理论的上界，这就导致了高复杂度的算法也有可能实际表现优秀（比如单纯形算法）。当然，在程设课程中，出题助教都会试图加强数据强度来卡掉高复杂度的做法，大家还是不要存在侥幸心理 **awa**。

总而言之，程序性能的估测需要结合复杂度和常数综合考虑，必要的时候，自己动手测一测程序的运行时间，往往比分析更加可靠。~~比如笔者曾经在竞赛中遇见了 $n \leq 10^6$ 但是 $O(n \log^2 n)$ 能通过~~的题，~~但是因为没敢写这个做法而遗憾离场。~~

空间复杂度

内存空间

当程序运行的时候，程序的机器码、程序使用的变量等数据均需要消耗内存空间。大家知道，计算机底层的数据储存形式是高 / 低电平，对应到 1 / 0，这被称为一个**位（bit）**。而 8 个位被称做一个**字节（byte）**，在现代计算机架构中，均以字节作为最小的数据操作单位。在谈论空间消耗的时候，一个字节记做 1B，我们还定义 1KiB = 1024B, 1MiB = 1024KiB, 1GiB = 1024MiB。

接下来考虑每年的程设助教都会遇到的简单的代码：

```

1  int a[1000000000000];
2  int main() {
3      ...
4  }

```

这个程序要运行起来需要消耗多大的空间呢？在通常的机器上，`int` 类型是**32 位有符号整数**，这意味着一个 `int` 需要 4 字节的储存空间。这里的数组大小足足有 10^{12} ，消耗的空间就是：

$$4\text{B} \times 10^{12} = 4 \times 10^{12}\text{B} \approx 3725\text{GiB}$$

这个空间消耗是远远大于大多数同学的电脑内存的，显然这个程序无法正常地运行起来。

空间消耗的复杂度表示

类似于时间复杂度，我们同样可以定义程序的空间复杂度，用来表示程序消耗的空间的量级。

比如，我们要储存一个长度为 n 的数组，程序的空间复杂度就是 $O(n)$ 的。如果我们要储存一个 $n \times n$ 的矩阵，程序的空间复杂度就是 $O(n^2)$ 的。

当然，在目前的程设课程中，大家都习惯于定义一个长度足够大的静态数组。但是在实际的工程代码中，这种做法通常是不可采取的，我们会按需进行动态分配。我们在计算空间复杂度中“消耗的空间”时，通常指的是随着问题规模 n 增大，实际上需要的 / 用到的空间的大小。不能说开了一个固定长度的静态数组，就代表空间复杂度 $O(1)$ 。

附录：渐进符号的严格定义

这里包括一点点数学分析的内容，用到的知识不会很难，都是大一初就会学的极限相关内容，想了解的可以自行学习，不想了解的不用管。

设 $[0, +\infty)$ 上的 $f(x), g(x)$ 满足 $f(x) > 0, g(x) > 0$ ：

- 若 $\lim_{x \rightarrow +\infty} \frac{f(x)}{g(x)} = L \neq 0$ ，那么称 $f(x) = \Theta(g(x))$ 。
- 若 $\lim_{x \rightarrow +\infty} \frac{f(x)}{g(x)} = L$ ，那么称 $f(x) = O(g(x))$ 。
- 若 $\lim_{x \rightarrow +\infty} \frac{f(x)}{g(x)} = 0$ ，那么称 $f(x) = o(g(x))$ 。
- 若 $\lim_{x \rightarrow +\infty} \frac{g(x)}{f(x)} = L$ ，那么称 $f(x) = \Omega(g(x))$ 。
- 若 $\lim_{x \rightarrow +\infty} \frac{g(x)}{f(x)} = 0$ ，那么称 $f(x) = \omega(g(x))$ 。

你应该可以看出来，这些定义是在比较 $f(x), g(x)$ 在“量级”上的大小，我们可以理解为：

- $f(x) = \Theta(g(x))$ 意味着在 x 充分的时候， $f(x), g(x)$ 存在某个固定的比例，也就是说在量级上 $f(x) = g(x)$ 。
- $f(x) = o(g(x))$ 有极限 $L = 0$ ，而 $L = 0$ 意味着在 x 充分大的时候，量级上 $f(x) < g(x)$ 。
- $f(x) = O(g(x))$ 综合了上面的两点，可以认为表示量级上 $f(x) \leq g(x)$ 。
- $f(x) = \omega(g(x))$ 则是反过来，它表示量级上 $f(x) > g(x)$ 。
- $f(x) = \Omega(g(x))$ 也可以用同样的方式分析，它表示 $f(x) \geq g(x)$ 。